课程笔记：操作电脑的 agent

Bowen-mantou（整理）

2026-10-23

视频作者/频道：B 站「常青藤名校课」· CMU 11-768 AI Agents（英文原声字幕版）
发布日期：2026-09-09
视频时长：74:26
视频链接：https://www.bilibili.com/video/BV1a6Yx62EH4?p=21
PDF 制作 Skill：https://github.com/wdkns/wdkns-skills

目录

1 引言：什么是 computer use agent（CUA） 2

1.1 定义：在人类使用的同一块屏幕上行动 2

1.2 为什么 CUA 特别难：它不是一个「可验证任务」 2

1.3 本章小结 2

2 历史：从玩具环境到真实环境 3

2.1 本章小结 3

3 评测：从静态动作到长程工作流 3

3.1 静态评测：在固定截图上判断动作对不对 4

3.2 端到端评测（web）：跑完整个任务再检查 4

3.3 端到端评测（桌面/移动） 4

3.4 长程评测：从几分钟到几小时 5

3.5 人类评测：仍然是金标准 5

3.6 本章小结 5

4 建模：VLM + 交错的视觉历史 5

4.1 动作空间与坐标表示 6

4.2 为什么是截图，而不是 HTML / 可访问性树 6

4.3 工程细节：加载与等待 6

4.4 本章小结 6

5 训练：预训练 grounding → SFT → RL 7

5.1 预训练：学会「看懂界面 + 定位元素」 7

5.2 后训练（SFT）：模仿人类与合成轨迹 7

5.3 后训练（RL）：让模型在模拟环境里自己试 7

5.4 本章小结 8

6 还剩下什么：速度、个性化、基础设施与多 agent 8

6.1 本章小结 8

7 总结与延伸 8

7.1 主讲人的收尾 8

7.2 把本讲压缩成六句话 9

7.3 延伸思考 9

7.4 拓展阅读（课程中提到的文献，供进一步查阅） 10

1 引言：什么是 computer use agent（CUA）

本讲由客座讲师 **Jing Yu Koh** 主讲（CMU 博士生，曾在 Google Research 与前沿实验室做 computer use agent 相关研究，也是 WebArena 等工作的作者之一），由课程主讲人 Daniel 介绍。主题是**操作电脑的 agent**（computer use agent, CUA）。

1.1 定义：在人类使用的同一块屏幕上行动

CUA 与前面几讲看到的 agent 有一个本质区别：**它的输入是图形用户界面（GUI）**，而不是文本环境或代码仓库。屏幕上可能是浏览器（web agent）、桌面（desktop agent），也可能是手机（mobile agent）；它们共同点是——**和人类一样看截图、做点击/滚动/输入/滑动这类动作**。

主讲人把 CUA 的工作方式概括成一个循环：

1. 给 agent 一个**用户任务**（自然语言）与**环境观测**（通常是一张截图，也可带其他文本表示）；
2. 模型（通常是视觉语言模型 VLM）产生一段**推理轨迹**，并输出**下一个动作**（点击、滚动、输入等）；
3. 环境**执行**该动作，返回一张新的截图；
4. 模型基于更新后的观测再次产生动作……直到模型认为完成、或预算耗尽；
5. 最后由环境、人类或另一个 judge 给出奖励（通常是 0/1，也有部分奖励）。

主讲人用「买一个蓝色马克杯」举例：模型看到杯子的截图，输出「我看到中间有个蓝色马克杯，我要点它」，格式化成一次左键点击；执行后得到细节页截图；模型再说「配送信息在下面，往下滚」，继续循环。这就是 ReAct 式的 **observe–reason–act** 循环。

本讲路线图

CUA 是什么（观察-推理-动作循环）→ 历史（从玩具环境到真实环境）→ 评测（静态 / 端到端 web / 端到端桌面 / 长程 / 人类评测）→ 建模（VLM + 交错视觉历史）→ 训练（预训练 grounding、SFT、RL）→ 还剩下什么（速度、个性化、基础设施、多 agent）。

1.2 为什么 CUA 特别难：它不是一个「可验证任务」

数学与代码有客观 verifier（符号判定、单元测试），而 computer use 在很多情况下**连人类之间都不一定同意「做没做成」**。教授在讲评测时反复强调这一点：成功与否往往主观、开放，很难写程序化 verifier。这直接导致 CUA 的评测体系与前面几讲完全不同——它必须组合**程序化检查**、**LLM judge**、**人类评测**等多种手段。

1.3 本章小结

CUA 是直接在图形界面上操作的 agent：观测截图、输出点击/滚动/输入等动作、观察新截图，循环直到完成或预算耗尽；奖励来自环境/人类/judge。它的根本难点在于任务不可完全形式化验证，这决定了后面评测与训练的特殊设计。

2 历史：从玩具环境到真实环境

主讲人用一条时间线讲清 CUA 的演进，这条线索对理解「为什么今天的 benchmark 长这样」很有帮助。

1. **2017：World of Bits / Mini World of Bits（OpenAI）**。这是最早的 CUA 论文（按 AI 的标准看非常古老）。环境是非常简化的合成网页，像手机界面但极不真实；模型用的是类似游戏 agent 的 RL 训练，而不是今天的大模型 RL。用主讲人的话说，这个项目「超前于时代」——当时的模型在这么简单的任务上都做得很差。

2. **2022：WebShop**。模拟购物环境，仍然人工，但比 Mini World of Bits 真实得多：模型在模拟的电商站点里点击、搜索、购买。此时语言模型 agent 开始表现得更好，训练方式也不再是重量级 RL。

3. **2024：WebArena 与 OSWorld**。WebArena 由 CMU 的 Graham 实验室等提出，用真实网站的数据（Amazon、Reddit 等）搭建了一组可自托管的模拟环境；OSWorld 进一步转向**真实桌面**（Linux/Windows 上的真实应用与真实数据），更通用。这两个基准是「模型开始变好」的转折点。

4. **2025–2026：环境被生成出来**。coding agent 变强之后，一批 benchmark 直接用 **LLM coding agent 端到端生成应用与网站**，并用**全合成数据**填充。幻灯片展示了一个 Gmail 克隆：不是真 Gmail，但看起来非常接近，完全由 coding agent 生成，数据全合成。主讲人认为这类环境有时比真实应用更好用——更干净、噪声更少。

5. **CoolWorld（CMU）**：仍然用真实软件，但偏「长尾」——天文学、生物学等专业软件，数据来自真实使用场景。它把 CUA 从消费级应用推向专业桌面软件。

6. **现在：GPT-6 Astra 的演示**。影片展示了它操作 CAD 软件做 3D 建模，演示被加速了 30 倍——也就是说实际速度慢得多、成本也高。但能力上，主讲人说它「基本刷爆了大多数 computer use benchmark」，在很多任务上超过人类基线。

从 2017 年的玩具环境到今天的专业软件操作，九年时间 CUA 从「模型完全做不了」走到「接近或超过普通人类」。主讲人给出的判断是：**能力上已经很强，但速度与成本仍是未解问题**。

2.1 本章小结

CUA 的历史分四个阶段：2017 玩具环境（Mini World of Bits）→ 2022 更真实的模拟购物（WebShop）→ 2024 真实数据环境与真实桌面（WebArena、OSWorld）→ 2025–2026 由 coding agent 端到端生成的环境与合成数据，以及专业软件（CoolWorld）。今天的模型能力已很强，瓶颈转到速度与成本。

3 评测：从静态动作到长程工作流

CUA 的评测比数学/代码复杂得多。主讲人把评测分成四类，并用一句话概括它们的差异：**从「预测一个动作」到「完成一个很长的工作流」**。图 **??** 是这四类的总览。

3.1 静态评测：在固定截图上判断动作对不对

静态评测（static evaluation）只看轨迹中的**单步动作是否正确**，不真正执行、不能端到端 rollout。图 **??** 是代表工作 **ScreenSpot-Pro**：它测的是模型的「grounding」能力——把「这个按钮/这个元素」映射到屏幕坐标。数据集包含 1,581 张截图、23 个应用、3 种操作系统，覆盖的各种专业软件界面（CAD、创意、办公、科学计算等）分辨率极高；评测标准是 **Point / region matching**：预测的坐标落在标注框内就算对。

另一个静态基准是 **Mind2Web**：给一张截图和一个任务（例如「在布鲁克林中心租车」），要模型预测**下一个动作**（例如点击 CAR 标签）；给定的是离线的截图序列与真值动作，不执行任何动作。它的规模是 2,350 个任务、137 个网站、31 种动作类型。

静态评测的优点是**快、便宜**：不需要跑环境，只需要截图和真值。缺点是**很多「假阴性」**：同一个状态往往有多个正确动作（比如不一定非要点 CAR 标签，先改日期也是合理的），但静态评测只有一个真值集合，会把其他合理动作判错。更重要的是，它**不测量我们真正关心的东西——端到端能不能完成任务**。

3.2 端到端评测（web）：跑完整个任务再检查

端到端评测给 agent 一个任务，让它在（模拟）环境里真正执行，最后用**类单元测试的程序化检查**判定是否完成。图 **??** 是代表工作 **WebArena**：812 个长程任务、4 个自托管的网站域（含 GitLab、Reddit、电商等），agent 在模拟环境里操作，结束时用一个**程序化 verifier** 检查环境最终状态（例如「repo 是否创建成功」）。

主讲人补充了一些很实在的细节：这些 verifier 是 CMU 的研究生**为每个任务手写**的；它们仍然有假阳性和假阴性（verifier 不够完备），也会带来 **reward hacking**——agent 可能去迎合 verifier 而不是真正完成任务。但它相比静态评测是巨大的进步：**在沙箱里测最终结果**，不监督每一步。

VisualWebArena 是 WebArena 的多模态扩展（图 **??**）：910 个任务、3 个站点，其中 25.2% 的任务需要图像输入才能表达（比如「帮我卖掉这张图里的商品」「像图里那样买最便宜的彩色照片打印机」）。它的评测既有程序化检查，也有多模态检查；主讲人的一句话总结是：**视觉理解改变了「哪些任务可以被表达」**。

当环境无法提供程序化验证（例如在真实网站上操作），就只能用 **LLM/VLM judge**：把 agent 的完整轨迹（动作 + 截图）和任务一起交给一个 judge 模型，让它判断是否成功。图 **??** 的 **WebVoyager** 是代表：643 个任务、15 个**真实网站**，评测完全依赖 VLM judge（幻灯片给出的 judge 与人类一致率是 85.3%）。它把 agent 放到真实 web 上，因此**真实感最高、可复现性最低**。

Online-Mind2Web 是 WebVoyager 的后续：300 个任务、136 个真实网站，judge 更复杂、与人类一致率 85.7%。它揭示了一个非常重要的现象——**benchmark drift**（图 **??**）：同一个模型，网站自报的成功率与人工评测的成功率差距很大，某些模型差出 26%–59%。也就是说，**真实网站的 benchmark 会随时间漂移，judge 的质量本身就是分数的一部分**。主讲人还提到：真实网站评测有一个现实问题——网站所有者不喜欢一堆 agent 来打流量，所以现在大家越来越少在真实 web 上跑 benchmark。

3.3 端到端评测（桌面/移动）

图 **??** 的 **OSWorld** 是桌面 CUA 的经典基准：一台真实的 Linux 虚拟机、9 个应用（GIMP、LibreOffice、Chrome 等）、369 个任务，每个任务都带**自定义的末状态检查**（例如「Excel 表格里的数字是否正确」「文件是否保存」）。任务会跨多个应用：从文件夹里找收据、读出金额、填进表格、

保存。

同类的还有 **Windows Agent Arena**（微软，154 个任务、11 个应用、平均约 20 分钟，并行 VM 把多任务评测做成了云规模的批处理）。它们的共同点是：**用程序化 verifier 检查最终状态**，仍然有假阳性/假阴性的问题，但比看图打分的 judge 更稳。

3.4 长程评测：从几分钟到几小时

到 2024–2025 年，主流 benchmark 大体被解决，社区开始转向**长程**（long horizon）：任务不再是十几分钟，而是人类要花几小时，甚至专门的专业软件工作流。图 **??** 的 **Odysseys**（CMU）是长程 web 的代表：200 个真实网页任务，每个任务平均 **6.1 条 rubric**，主评测上限 100 步。例如「为棕榈泉的婚礼规划行程」需要查往返航班、查 9 点到 16 点的车程、订酒店、租车、评估绕路、用 CryptPad 生成日程——成功不是单一的 0/1，而是**逐条 rubric 打分**（例子里是 7/8 通过，得分 88%）。

OSWorld 2.0 则把长程推到桌面（图 **??**）：108 个工作流，人类中位耗时 **1.6 小时**，一个 agent 平均要 **318 次调用**；评测是**部分检查 + 二值检查**的组合。幻灯片的结论句很关键：**前沿从「点击」转向隐藏状态、变化的需求与验证**——任务例如填报销单：读教程 PDF、在多个系统里填表、从收据里挑金额、发邮件、中途还要处理「乘客信息有误，要不要报销」这样的追问。

此外还有 **CoolWorld-Long**（CMU）：200 个专业软件长程任务，多数需要 500 次以上的 LLM 调用；发布时模型表现很差，但主讲人说包括 GPT-6 在内的最新模型已经接近 90%。

3.5 人类评测：仍然是金标准

主讲人强调：无论程序化 verifier 还是 LLM judge，都只是**代理指标（proxy）**；真正可靠的仍是人类评审——让几个人看轨迹、独立投票判断 agent 有没有完成任务。很多论文都做人类基线来校准 judge（幻灯片给的一致率在 85%–90% 量级）。但人类评测**非常贵**，所以通常只在 benchmark 发布时做一两次、用来验证 judge 的可靠性；日常 debug 与验证不可能每次都请人。

3.6 本章小结

CUA 评测有四类：静态（ScreenSpot-Pro、Mind2Web，快但假阴性多、不测端到端）、web 端到端（WebArena/VisualWebArena 程序化检查；WebVoyager/Online-Mind2Web 用 judge）、桌面端到端（OSWorld、Windows Agent Arena）、长程（Odysseys 的 rubric、OSWorld 2.0 的部分 + 二值检查）。程序化检查与 LLM judge 都是代理，人类评审仍是金标准；真实网站评测存在 drift，并且会打扰网站。

4 建模：VLM + 交错的视觉历史

讲完评测，主讲人转向「这些模型是怎么工作的」。CUA 的建模比想象中简单：**它就是一个接受交错视觉/动作历史的视觉语言模型**。

图 **??** 是最基本的策略表示：给定目标（goal）与当前截图（o_0），模型输出动作 a_0：

$$\pi_\theta(a_0 \mid \text{goal}, o_0).$$

例子是「买一个 20 美元以下的蓝色马克杯」+ 一张商品列表截图 → 输出 `click`(x_0, y_0)。

随着任务推进，上下文变成**交错序列**（图 **??**）：

$$\pi_\theta(a_t \mid \text{goal}, o_{\le t}, a_{<t}),$$

即目标 + 所有历史观测 + 所有历史动作。幻灯片给出了 token 化后的样子：goal tokens、visual tokens（第 0 轮）、action tokens（第 0 轮）、visual tokens（第 1 轮）、action tokens（第 1 轮）……最后模型输出第 t 步的动作 token（`click` (x_t, y_t)）。文本走 tokenizer + embedding，截图走 vision encoder + projector（投影到语言模型的 token 空间）。

4.1 动作空间与坐标表示

不同模型的动作空间略有不同，但「眯着眼看」都差不多：一组点击/滚动/输入等动作，用文本格式（如 PyAutoGUI 风格、tool call、XML）表示。主讲人提到，各家的格式与 schema 不同，**这本身就是可复现性的噩梦**（下一节训练时还会强调这一点）。

坐标有一个重要的工程约定：通常**归一化到 0–1000**（(0,0) 是左上角，(1000,1000) 是右下角），而不是绝对像素。这样训练出的坐标能跨分辨率迁移——在 Mac 上训练，换到平板或其他分辨率也能用。

4.2 为什么是截图，而不是 HTML / 可访问性树

一个常见问题是：为什么不直接用 HTML 或 accessibility tree？主讲人的回答很坦率：

- **HTML 很脏**，但有简化版本（accessibility tree）可用；桌面端的可访问性表示并不统一，也不总是能拿到。
- 截图**更简单**、**更通用**：任何设备、任何应用都能截图；训练一个只看截图的模型，可以更直接地迁移到不同操作系统。
- 出乎很多人意料：**纯截图方案已经足够强**，强到「用结构化表示好像不是必须的」。一年前很多人还不相信这一点。

不过他也澄清：前沿模型并不是二选一——例如 OpenAI 的模型在可访问性表示可用时会使用它；多种表示结合可以更鲁棒。

4.3 工程细节：加载与等待

CUA 遇到大量「烦人的基础设施问题」，最典型的是**加载**：点击之后页面还没加载完，截到的图是加载中的。最常见的做法是**动作后等几秒再截图**（如 sleep 3 秒）；如果截到加载界面，模型被训练成「再等一会儿」。更复杂的做法是「等画面变化」，但主讲人说最简单、最常用的还是 sleep。

4.4 本章小结

CUA 的建模本质是一个 VLM：输入 goal 与交错的「截图—动作」历史，输出文本化的动作；坐标归一化到 0–1000 以便跨分辨率迁移；截图表示简单通用、出乎意料地强，但前沿模型也会结合可访问性表示；工程上要处理加载等待等边角问题。

5 训练：预训练 grounding → SFT → RL

主讲人把 CUA 的训练拆成三个阶段，并强调一个前提：**今天的前沿 CUA 通常不是「只会 CUA 的专用模型」，而是通用模型的一项能力**（例如 GPT-6 Astra 也会穿插 bash 调用、写代码）。所以 CUA 数据是**混进通用预训练/后训练**的，而不是单独训一个模型。

5.1 预训练：学会「看懂界面 + 定位元素」

预训练阶段的目标非常基础：让模型学会 **grounding**（把界面元素与坐标对应）和简单的动作预测。图 **??** 给了两类数据：

1. **UI elements / grounding**：给一张网页截图，训练模型预测某个元素的**边界框/屏幕坐标**（如「CAR」对应某个 element box）。
2. **Action prediction**：给截图与元素描述（如 element: bookCarTab, value: none），让模型预测动作（CLICK）；其他标签还有 type(text)、swipe(start, end) 等。

这些数据「噪声大但量大」：主讲人举了一个很具体的采集方法——在真实网站上跑一段 JavaScript，就能批量导出所有元素的边界框；把「car」这样的元素名与坐标配对，就是一条 grounding 数据。这类数据被大量混入前沿模型预训练的早期阶段，目的是让后面的 CUA 训练「更容易起步」。

5.2 后训练（SFT）：模仿人类与合成轨迹

有了基础能力之后，进入 SFT 阶段：让模型模仿有用的轨迹（图 **??**）。数据来自两类：

- **人类数据**：雇人做任务并记录「截图—动作」轨迹。例如 MolmoWebMix 收集了 **36K 条人类轨迹 + 105K 条合成轨迹**；AgentNet 发布了 **22,625 条**任务轨迹（跨 3 个操作系统）。这是最直接的「行为克隆」。
- **合成数据**：用**有特权信息**的模型来生成轨迹。例如给一个模型网站的 accessibility tree（简化 HTML），它在结构化空间里能做得比纯截图模型好得多；让它在这个「特权」条件下 rollout，收集轨迹，再过滤（用另一个 LLM judge 或人工筛掉质量差的），最后用它训练只看截图的模型。MolmoWebMix 就是这样生成了 105K 条合成轨迹。

主讲人用了一个很好的类比：这就像特斯拉在训练时给车装 LiDAR——LiDAR 让采集到的轨迹质量更高，然后训练时把 LiDAR 信息丢掉，只让模型学像素输入。合成数据的「特权信息」也是同样的角色：**采集时用强信息，部署时只靠截图**。

5.3 后训练（RL）：让模型在模拟环境里自己试

SFT 之后模型已经有基本的 CUA 能力，可以直接用 prompt 驱动；但真正带来大幅提升的是 **RL**（图 **??**）：把模型放进模拟环境，给它任务，让它自主试点，按最终奖励更新策略——**没有人类演示**。

主讲人给了一个具体的收益数字：SFT 之后成功率大约 **0.5**，做 RL 之后能爬到 **0.6 以上**。更重要的是，RL 能带来「人类演示很难教」的能力——比如长程任务：你很难为几小时的任务收集大量人类演示，但可以让模型在模拟环境里自己跑。

为什么一定要模拟环境？主讲人举了个很直白的例子：你只能在 Uber 上真正订车有限次，多了会被封号或破产；更不用说重复尝试会打扰真实的人、产生不可撤销的副作用。因此 RL 需要**可重置**、**可注入状态**、**可检查最终状态**的模拟环境。

图 **??** 是这类基础设施的代表 **CUA-Gym**：它用 plan agent / dev agent / web agent 等流程，构建了 **94 个可复用的 mock web 应用**——每个应用都支持**注入初始状态**、**检查最终状态**、**每次会话重置**。幻灯片另一页还提到「co-generate tasks, states, and rewards」(-32K verified tuples / 110 environments)：让环境、任务与奖励一起生成，再用「有特权信息的 judge + 加权检查清单」验证，从而把任务规模做上去。

5.4 本章小结

CUA 训练分三阶段：预训练用大规模噪声 grounding/动作预测数据打基础；SFT 用人类演示 + 特权信息模型生成的合成轨迹做行为克隆；RL 把模型放进可重置的模拟环境自主试错，用 verifier/rubric/judge 打分。RL 能把成功率从约 0.5 提到 0.6+，并能训练人类演示难以覆盖的长程能力；CUA-Gym 这类基础设施是 RL 的前提。

6 还剩下什么：速度、个性化、基础设施与多 agent

讲到最后，主讲人没有回避「还有什么没解决」，而是把剩余问题列得很清楚：

1. **速度与成本**。任务耗时大致可以写成「轮数 × 每轮耗时」。前沿 CUA 模型能力很强，但每个动作往往要一次完整的 VLM 推理，导致实际运行非常慢、非常贵（演示里 30 倍速播放就是这个原因）。幻灯片还给了几条「更快」的模型设计方向：推理只在关键决策处做（Game-TARS 的稀疏思考）、直接预测动作而不做完整推理（TOM-1 式直接动作预测）——用更少的思考换速度。

2. **个性化**。每个人都希望 agent 按自己的习惯操作电脑；这需要模型理解用户的偏好、历史与隐私边界，是一个开放的研究方向。

3. **基础设施与 UX**。大规模评测与训练需要 VM 池、并行调度、状态重置、可观测性；这些工程问题与平台团队的日常高度重合。

4. **多 agent**。主讲人提到 MACU（上一讲讲过）的调度器结构：**plan → parallelize → replan**。关键结论是：**并行只有在「协调不破坏正确性」时才有帮助**——子任务之间如果存在隐藏依赖，并行反而会出错。

6.1 本章小结

CUA 的能力已接近「解决」，但仍有四类问题：速度与成本（每轮一次 VLM 推理）、个性化、基础设施与 UX（VM 池/重置/可观测）、多 agent（plan → parallelize → replan，并行必须不破坏协调正确性）。

7 总结与延伸

7.1 主讲人的收尾

主讲人用一页总结收束全讲：

本讲总结（据主讲人收尾整理）

1. **CUA 与其他 agent 不同**：它直接在人类使用的同一套 GUI 上操作，输入是截图、输出是点击/滚动/输入等动作。
2. **评测很棘手**：从静态动作到长程工作流，任务与指标差异巨大；程序化 verifier 与 LLM judge 都只是代理，人类评审仍是金标准。
3. **建模范式很简单**：用一个 VLM，从交错的「目标—截图—动作」历史中预测下一个动作；坐标归一化以支持跨设备迁移。
4. **训练是分阶段的**：大规模噪声预训练（grounding）→ SFT（人类演示 + 合成轨迹）→ RL（模拟环境自主 rollout + 奖励）。
5. **仍然有很多可做**：速度、个性化、基础设施/UX、多 agent。

7.2 把本讲压缩成六句话

本讲核心结论（背这几句）

1. CUA = 看截图、做人类动作（点击/滚动/输入）的 VLM agent；observe–reason–act 循环持续到完成或预算耗尽。
2. 它的核心难点是**不可完全形式化验证**：同一任务可以有多种正确做法，成功与否有时连人类都不一致。
3. 评测四类：静态（快但假阴性多）、web 端到端（程序化 verifier）、桌面/移动端到端（VM + 末状态检查）、长程（rubric/部分奖励）；judge 是代理，人类是金标准。
4. 建模：$\pi_\theta(a_t \mid \text{goal}, o_{\le t}, a_{<t})$，goal 文本 + 截图视觉 token + 动作 token 交错；坐标归一化 0–1000；截图简单通用，可访问性表示可作补充。
5. 训练三阶段：预训练 grounding → SFT 行为克隆（人类 + 特权信息合成）→ RL 模拟环境自主试错；RL 把成功率从约 0.5 提到 0.6+。
6. 今天能力已接近解决，但速度/成本、个性化、基础设施与多 agent 协调仍是开放问题。

7.3 延伸思考

1. 「**不可验证**」**决定了整套设计**。因为 CUA 任务不像数学/代码那样有客观 verifier，评测必须用 rubric、judge、人类多重代理，训练必须依赖模拟环境与部分奖励。这与前面 coding agent 那一讲的「测试质量决定训练上限」是同一枚硬币的两面：**验证信号的可靠程度，决定了 agent 能力的上限**。
2. **rubric 与人类校准是评测工程的核心**。Odysseys 与 Online-Mind2Web 都花大量篇幅在「judge 与人类一致率」（85
3. **特权信息合成是数据杠杆**。用 accessibility tree 教师生成轨迹、再用纯截图模型学习——这和「用强模型蒸馏弱模型」同构。关键不是数据量，而是**教师能看到学生看不到的信息**，且推理时不能依赖这些信息。

4. **模拟环境就是 CUA 的「测试集 + 训练场」**。与 coding agent 的 SWE-Gym/SWE-rebench 一样，CUA-Gym 这类工作把「环境」变成可生成、可重置、可注入状态的资产。平台团队的 VM 池、调度、快照能力，直接决定 RL 的规模。

5. **速度是产品化的硬约束**。每轮一次 VLM 推理意味着长任务成本极高；稀疏思考、直接动作预测、缓存与并行都是工程上的下一步。这也是「能力接近解决、产品仍未普及」的根本原因。

7.4 拓展阅读（课程中提到的文献，供进一步查阅）

- Mini World of Bits（2017，OpenAI）：最早的 CUA 环境；WebShop（2022）：模拟购物环境。
- WebArena、VisualWebArena（CMU）：可复现的仿真 web 环境与多模态扩展。
- WebVoyager、Online-Mind2Web：真实网站评测与 LLM/VLM judge 校准。
- OSWorld、OSWorld 2.0、Windows Agent Arena：真实桌面/Windows 端到端评测。
- ScreenSpot-Pro、Mind2Web：grounding 与下一步动作的静态评测。
- Odysseys、CoolWorld / CoolWorld-Long（CMU）：长程 web 与专业软件评测。
- MolmoWebMix、AgentNet：CUA 人类演示轨迹数据；特权信息合成数据。
- CUA-Gym、Gym-Anything：可重置的 CUA 训练环境与任务/状态/奖励生成。
- MACU（Multi-Agent Computer Use）：plan → parallelize → replan 的多 agent 电脑操作框架。